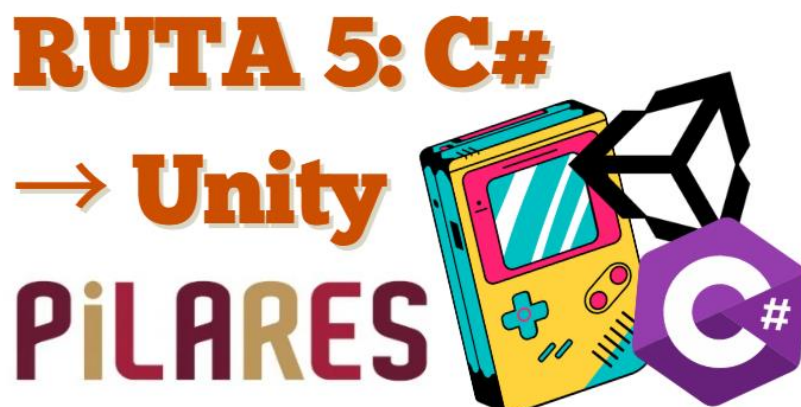


RUTA 5: C# → Unity



Descripción del Taller

Este taller está diseñado con un enfoque práctico y progresivo para el desarrollo de videojuegos 2D utilizando Unity y el lenguaje de programación C#. A lo largo de la ruta, las y los usuarios aprenderán desde los fundamentos de programación hasta la implementación de un videojuego completo, integrando mecánicas, físicas, interfaz de usuario y arquitectura de software, bajo un flujo de trabajo profesional.

Información general

- **Nivel:** Intermedio
- **Duración:** 160 horas, distribuidas en sesiones de 2 horas, 4 veces por semana.
- **Perfil de los Usuarios:**
 - Personas interesadas en el desarrollo de videojuegos que desean aprender programación en C# y su aplicación en la creación de videojuegos 2D dentro del motor Unity.

Categoría	Descripción
Conocimientos previos	Experiencia básica en programación.
Habilidades	- Pensamiento lógico y resolución de problemas. - Capacidad de seguir instrucciones en pasos secuenciales. - Disposición para aprender en comunidad y colaborar con otros. - Interés por el desarrollo de videojuegos. - Capacidad de análisis y organización de código.

Objetivo General

Desarrollar habilidades para diseñar e implementar videojuegos 2D en Unity, aplicando programación orientada a objetos en C#, integrando scripts, componentes, interfaz de usuario y arquitectura del videojuego bajo un flujo de trabajo profesional.

Objetivos particulares

Al finalizar su respectivo taller, las y los usuarios serán capaces de:

- Aplicar estructuras fundamentales de programación en C#.
- Diseñar sistemas mediante programación orientada a objetos.
- Implementar mecánicas de juego en Unity utilizando scripts.
- Comprender la arquitectura basada en componentes de Unity.
- Integrar físicas, colisiones e interacción en videojuegos 2D.
- Diseñar interfaces de usuario funcionales.
- Gestionar escenas, flujo del juego y estados.
- Desarrollar videojuegos completos con estructura modular y buenas prácticas.

Duración

El taller tiene una duración total de 160 horas, distribuidas en módulos progresivos que permiten el desarrollo gradual de habilidades desde la programación en C# hasta la construcción de un videojuego 2D completo en Unity.

Esta distribución permite:

- Un aprendizaje progresivo y práctico.
- Integración de programación y desarrollo en motor.
- Desarrollo de prototipos funcionales por módulo.
- Consolidación de un proyecto final completo.

Estructura del taller RUTA 5: C# → Unity Módulo 1 (40 horas - 20 sesiones de 2 horas)

Sesiones y temas	Subtemas	Resultado del aprendizaje	Metodología sugerida	Duración sugerida
Sesión 1: Tema 1. Tipos de datos y estructuras básicas	1.1 Tipos primitivos	Comprenderá tipos de datos.	Ejercicios prácticos de declaración de variables.	2 horas

Sesión 2: Tema 1. Tipos de datos y estructuras básicas	1.1.2 Inicialización	Inicializará variables.	Práctica con variables en consola.	2 horas
Sesión 3: Tema 1. Tipos de datos y estructuras básicas	1.1.3 Alcance de variables	Identificará scope.	Ejercicios de alcance de variables.	2 horas
Sesión 4: Tema 1. Tipos de datos y estructuras básicas	1.1.4 Conversión implícita y explícita	Aplicará casting.	Práctica de conversión de datos.	2 horas
Sesión 5: Tema 1. Tipos de datos y estructuras básicas	1.2 Diferencias entre float y double en físicas	Diferenciará precisión.	Ejercicios con cálculos de movimiento.	2 horas
Sesión 6: Tema 1. Tipos de datos y estructuras básicas	1.3 Constantes y casting	Usará constantes.	Implementación en cálculos simples.	2 horas
Sesión 7: Tema 1. Tipos de datos y estructuras básicas	1.4 Arreglos	Manipulará arrays.	Ejercicios con posiciones de arreglo.	2 horas
Sesión 8: Tema 1. Tipos de datos y estructuras básicas	1.5 List<T>	Gestionará listas.	Creación de listas dinámicas.	2 horas
Sesión 9: Tema 1. Tipos de datos y estructuras básicas	1.6 Diccionarios para inventarios simples	Implementará inventario.	Uso de clave–valor en consola.	2 horas
Sesión 10: Tema 2. Métodos y control de flujo	Tema 2. Métodos y control de flujo	Crearé funciones.	Crearé funciones.	2 horas
Sesión 11: Tema 2. Métodos y control de flujo	2.2 Parámetros por valor y referencia	Usará ref y out.	Práctica de paso de parámetros.	2 horas

Sesión 12: Tema 2. Métodos y control de flujo	2.3 Sobrecarga	Aplicará sobrecarga.	Diseño de métodos múltiples.	2 horas
Sesión 13: Tema 2. Métodos y control de flujo	2.4 Condicionales	Implementará decisiones.	Ejercicios con if–else.	2 horas
Sesión 14: Tema 2. Métodos y control de flujo	2.5 Switch	Usará switch.	Práctica con enums.	2 horas
Sesión 15: Tema 2. Métodos y control de flujo	2.6 Ciclos	Implementará loops.	Ejercicios con for, while, foreach.	2 horas
Sesión 16: Tema 2. Métodos y control de flujo	2.7 Manejo básico de excepciones	Controlará errores.	Uso de try–catch.	2 horas
Sesión 17: Tema 3. Introducción a POO	3.1 Clase y objeto	Comprenderá POO.	Creación de clases básicas.	2 horas
Sesión 18: Tema 3. Introducción a POO	3.2 Propiedades	Usará get/set.	Implementación de propiedades.	2 horas
Sesión 19: Tema 3. Introducción a POO	3.3 Constructores	Crearé constructores.	Inicialización de objetos.	2 horas
Sesión 20: Tema 3. Introducción a POO	3.4 Encapsulamiento	Aplicará control de acceso.	Uso de campos privados.	2 horas

Material de apoyo:

Documentación **oficial** **de** **C#**

<https://learn.microsoft.com/dotnet/csharp/>

Guía completa del lenguaje C#.

Tipos **de** **datos** **en** **C#**

<https://learn.microsoft.com/dotnet/csharp/language-reference/builtin-types/built-in-types>

Explica los tipos primitivos del lenguaje.

Colecciones **en** **C#**

<https://learn.microsoft.com/dotnet/standard/collections/>

Uso de listas, arreglos y diccionarios.

Métodos **en** **C#**

<https://learn.microsoft.com/dotnet/csharp/programming-guide/classes-and-structs/methods>

Declaración y uso de funciones.

Programación **Orientada** **a** **Objetos** **en** **C#**

<https://learn.microsoft.com/dotnet/csharp/fundamentals/object-oriented/>

Conceptos básicos de POO.

Evaluación

La evaluación del módulo se realizará mediante una actividad práctica integradora y un cuestionario teórico, con el propósito de valorar la comprensión de las estructuras fundamentales de programación en C#.

La actividad práctica consistirá en el desarrollo de un sistema en consola que simule un personaje de videojuego con atributos y comportamiento.

El cuestionario teórico permitirá evaluar conceptos como tipos de datos, estructuras, métodos, control de flujo y programación orientada a objetos.

La evaluación final se centrará en la actividad integradora.

Actividades de integración

Objetivo de la evaluación:

Valorar la capacidad de las y los usuarios para aplicar estructuras fundamentales de programación en C# en la construcción de sistemas básicos de videojuego.

Actividad Integradora (Evaluación)

Objetivo:	Evaluar la construcción de un sistema básico de personaje en consola.
Instrucciones:	Desarrolla la siguiente actividad de forma clara, estructurada y funcional. La evidencia deberá presentarse en formato imagen (.jpg o .png) o PDF con un peso no mayor a 100 kb. Desarrolla un sistema que incluya: • Clase Personaje con atributos (vida, ataque, defensa) • Inventario utilizando List o Dictionary • Método RecibirDaño() • Validación de muerte del personaje • Mensajes dinámicos en consola • Organización en múltiples clases (valor 50.0) El sistema debe ser funcional, organizado y correctamente estructurado. Adjunta capturas del código y ejecución en consola.

Estructura del taller RUTA 5: C# → Unity Módulo 2 (30 horas - 15 sesiones de 2 horas)

Sesiones y temas	Subtemas	Resultado del aprendizaje	Metodología sugerida	Duración sugerida
Sesión 1: Tema 1. Herencia y polimorfismo	1.1 Clase base Entidad	Comprenderá la jerarquía de clases.	Creación de clase base para videojuegos.	2 horas
Sesión 2: Tema 1. Herencia y polimorfismo	1.2 Clases derivadas	Implementará herencia.	Creación de Jugador y Enemigo.	2 horas
Sesión 3: Tema 1. Herencia y polimorfismo	1.3 Override	Aplicará sobrescritura.	Modificación de métodos heredados.	2 horas
Sesión 4: Tema 1. Herencia y polimorfismo	1.4 Polimorfismo aplicado a combate	Implementará comportamiento dinámico.	Simulación de combate con referencias base.	2 horas
Sesión 5: Tema 2. Clases abstractas e interfaces	2.1 Métodos abstractos	Comprenderá abstracción.	Creación de clases abstractas.	2 horas

Sesión 6: Tema 2. Clases abstractas e interfaces	2.2 Interfaces	Aplicará contratos.	Implementación de interfaces.	2 horas
Sesión 7: Tema 3. Simulación avanzada	3.1 Enum para estados	Definirá estados.	Uso de enum en lógica de juego.	2 horas
Sesión 8: Tema 3. Simulación avanzada	3.2 Máquina de estados	Controlará el comportamiento.	Implementación de estados del personaje	2 horas
Sesión 9: Tema 3. Simulación avanzada	3.3 Sistema por turnos	Organizará el flujo de juego.	Simulación de combate por turnos.	2 horas
Sesión 10: Tema 3. Simulación avanzada	3.4 Validación de victoria/derrota	Validará resultados.	Programación de victoria y derrota.	2 horas
Sesión 11	Integración	Integrará sistemas.	Desarrollo de simulación completa.	2 horas
Sesión 12	Práctica	Mejorará arquitectura.	Ajustes de código modular.	2 horas
Sesión 13	Práctica	Optimizará la estructura.	Refactorización de clases.	2 horas
Sesión 14	Evaluación	Desarrollará un sistema completo.	Integración final del sistema.	2 horas
Sesión 15	Cierre	Presentará proyecto.	Presentación y retroalimentación.	2 horas

Material de apoyo:

Herencia

en

C#

<https://learn.microsoft.com/dotnet/csharp/fundamentals/object-oriented/inheritance>

Explica cómo reutilizar código mediante herencia.

Polimorfismo

en

<https://learn.microsoft.com/dotnet/csharp/fundamentals/object-oriented/polymorphism>

Uso de comportamiento dinámico en objetos.

Clases**abstractas**

e

interfaces<https://learn.microsoft.com/dotnet/csharp/programming-guide/classes-and-structs/abstract-and-sealed-classes-and-class-members>

Conceptos de abstracción y contratos.

4.**Enumeraciones****(enum)**<https://learn.microsoft.com/dotnet/csharp/language-reference/builtin-types/enum>

Uso de estados definidos en el código.

5.**Buenas****prácticas**

en

P00<https://learn.microsoft.com/dotnet/standard/design-guidelines/>

Guía de diseño de código estructurado.

Evaluación

La evaluación del módulo se realizará mediante una actividad práctica integradora y un cuestionario teórico, con el propósito de valorar la implementación de sistemas estructurados mediante programación orientada a objetos.

La actividad práctica consistirá en el desarrollo de una simulación de combate entre múltiples entidades con comportamiento dinámico.

El cuestionario teórico permitirá evaluar conceptos como herencia, polimorfismo, abstracción, interfaces y control de estados.

La evaluación final se centrará en la actividad integradora.

Actividades de integración**Objetivo de la evaluación:**

Valorar la capacidad de las y los usuarios para diseñar sistemas de videojuego utilizando programación orientada a objetos y estructuras escalables en C#.

Actividad Integradora (Evaluación)

Objetivo:	Evaluar la construcción de un sistema estructurado de combate en consola.
Instrucciones:	Desarrolla la siguiente actividad de forma clara, organizada y funcional. La evidencia deberá presentarse en formato imagen (.jpg o .png) o PDF con un peso no mayor a 100 kb.

	Desarrolla un sistema que incluya: • Clase abstracta Entidad • Clases derivadas (Jugador y múltiples enemigos) • Uso de polimorfismo en el método Atacar() • Implementación de enum para estados • Sistema de combate por turnos • Validación automática de victoria o derrota (valor 50.0) El sistema debe ser funcional, modular y correctamente estructurado. Adjunta capturas del código y ejecución en consola.
--	--

Estructura del taller RUTA 5: C# → Unity Módulo 3 (40 horas - 20 sesiones de 2 horas)

Sesiones y temas	Subtemas	Resultado del aprendizaje	Metodología sugerida	Duración sugerida
Sesión 1: Tema 1. Instalación y configuración profesional	1.1 Unity Hub	Comprenderá instalación.	Instalación guiada de Unity Hub.	2 horas
Sesión 2: Tema 1. Instalación y configuración profesional	1.2 Instalación LTS	Configurará el entorno estable.	Instalación de versión LTS.	2 horas
Sesión 3: Tema 1. Instalación y configuración profesional	1.3 Creación de proyecto 2D	Crearé proyecto.	Configuración inicial de proyecto.	2 horas
Sesión 4: Tema 1. Instalación y configuración profesional	1.4 Integración con Visual Studio	Conectará al entorno.	Configuración con Visual Studio.	2 horas
Sesión 5: Tema 2. Interfaz completa	2.1 Hierarchy	Comprenderá estructura.	Organización de objetos en escena.	2 horas

Sesión 6: Tema 2. Interfaz completa	2.2 Scene	Manipulará el entorno.	Uso de herramientas 2D.	2 horas
Sesión 7: Tema 2. Interfaz completa	2.3 Game	Ejecutará proyecto.	Ajuste de resolución y ejecución.	2 horas
Sesión 8: Tema 2. Interfaz completa	2.4 Inspector	Configurará los componentes.	Modificación de propiedades.	2 horas
Sesión 9: Tema 2. Interfaz completa	2.5 Project	Organizará los archivos.	Creación de carpetas profesionales.	2 horas
Sesión 10: Tema 2. Interfaz completa	2.6 Console	Depurará errores.	Uso de Debug.Log.	2 horas
Sesión 11: Tema 3. Arquitectura y programación básica	3.1 GameObjects y componentes	Comprenderá componentes.	Creación de objetos con scripts.	2 horas
Sesión 12: Tema 3. Arquitectura y programación básica	3.2 Transform	Manipulará la posición.	Ejercicios de movimiento básico.	2 horas
Sesión 13: Tema 3. Arquitectura y programación básica	3.3 Prefabs	Reutilizará objetos.	Creación e instanciación.	2 horas
Sesión 14: Tema 3. Arquitectura y programación básica	3.4 Ciclo de vida	Comprenderá el flujo.	Uso de Awake, Start, Update.	2 horas
Sesión 15: Tema 3. Arquitectura y programación básica	3.5 Movimiento lateral	Implementará control.	Movimiento con Input.GetAxis.	2 horas

Sesión 16	Integración	Integrará sistemas.	Desarrollo de prototipo.	2 horas
Sesión 17	Práctica	Mejorará la estructura.	Ajustes de código y organización.	2 horas
Sesión 18	Práctica	Optimizará el flujo.	Mejora del script.	2 horas
Sesión 19	Evaluación	Desarrollará un prototipo.	Integración final.	2 horas
Sesión 20	Cierre	Presentará proyecto.	Retroalimentación.	2 horas

Material de apoyo:

Unity Hub y gestión de versiones

<https://docs.unity3d.com/hub/manual/index.html>

Guía para instalar y gestionar versiones de Unity.

GameObjects

y

componentes

<https://docs.unity3d.com/Manual/GameObjects.html>

Sistema basado en componentes de Unity.

Ciclo

de

vida

de

scripts

<https://docs.unity3d.com/Manual/ExecutionOrder.html>

Funcionamiento de Awake, Start y Update.

Input

en

Unity

<https://docs.unity3d.com/ScriptReference/Input.GetAxis.html>

Captura de controles para movimiento.

Evaluación

La evaluación del módulo se realizará mediante una actividad práctica integradora y un cuestionario teórico, con el propósito de valorar el uso del entorno Unity y la implementación de mecánicas básicas mediante scripts.

La actividad práctica consistirá en el desarrollo de un prototipo 2D funcional con movimiento y estructura organizada.

El cuestionario teórico permitirá evaluar conceptos como interfaz de Unity, GameObjects, componentes, ciclo de vida y entrada de usuario.

La evaluación final se centrará en la actividad integradora.

Actividades de integración

Objetivo de la evaluación:

Valorar la capacidad de las y los usuarios para utilizar el entorno de Unity y desarrollar mecánicas básicas de videojuego mediante scripts en C#.

Actividad Integradora (Evaluación)

Objetivo:	Evaluar el desarrollo de un prototipo 2D funcional en Unity.
Instrucciones:	Desarrolla la siguiente actividad de forma clara, organizada y funcional. La evidencia deberá presentarse en formato imagen (.jpg o .png) o PDF con un peso no mayor a 100 kb. Desarrolla un prototipo que incluya: • Personaje con movimiento lateral fluido • Uso correcto de prefab • Organización profesional del proyecto (carpetas y jerarquía) • Script funcional conectado al objeto • Uso de Debug.Log para validación (valor 50.0) El sistema debe ser funcional, organizado y correctamente estructurado. Adjunta capturas del entorno Unity, código y ejecución del prototipo.

Estructura del taller RUTA 5: C# → Unity Módulo 4 (40 horas - 20 sesiones de 2 horas)

Sesiones y temas	Subtemas	Resultado del aprendizaje	Metodología sugerida	Duración sugerida
Sesión 1: Tema 1. Física aplicada al videojuego	1.1 Rigidbody2D	Comprenderá físicas.	Implementación de físicas en personaje.	2 horas
Sesión 2: Tema 1. Física aplicada al videojuego	1.2 Sistema de colisiones	Detectará impactos.	Configuración de colliders.	2 horas
Sesión 3: Tema 1. Física aplicada al videojuego	1.3 Triggers	Implementará eventos.	Uso de IsTrigger	2 horas

Sesión 4: Tema 1. Física aplicada al videojuego	1.4 Detección de impactos	Validará colisiones.	Programación de eventos de impacto.	2 horas
Sesión 5: Tema 1. Física aplicada al videojuego	1.5 Movimiento con físicas	Aplicará salto	Implementación de salto y gravedad.	2 horas
Sesión 6: Tema 2. Interfaz de usuario	2.1 Canvas	Crearé UI.	Configuración de interfaz.	2 horas
Sesión 7: Tema 2. Interfaz de usuario	2.2 Text / Image	Mostrará información.	UI dinámica desde script.	2 horas
Sesión 8: Tema 2. Interfaz de usuario	2.3 Button	Controlará eventos.	Programación de botones.	2 horas
Sesión 9: Tema 3. Gestión de flujo del juego	3.1 SceneManager	Gestionará escenas.	Cambio de escenas.	2 horas
Sesión 10: Tema 3. Gestión de flujo del juego	3.2 Menú principal	Diseñará navegación.	Creación de menú inicial.	2 horas
Sesión 11: Tema 3. Gestión de flujo del juego	3.3 Pantalla final	Mostrará resultados.	Implementación de victoria/derrota.	2 horas
Sesión 12: Tema 4. Organización avanzada	4.1 Comunicación entre scripts	Conectará scripts.	Uso de GetComponent.	2 horas
Sesión 13: Tema 4. Organización avanzada	4.2 Separación de responsabilidades	Organizará lógica.	Scripts por responsabilidad.	2 horas

Sesión 14: Tema 4. Organización avanzada	4.3 Buenas prácticas	Mejorará código.	Refactorización básica.	2 horas
Sesión 15	Integración	Integrará sistemas.	Desarrollo del videojuego.	2 horas
Sesión 16	Práctica	Mejorará mecánicas.	Ajustes de jugabilidad.	2 horas
Sesión 17	Práctica	Optimizará el sistema.	Mejora del rendimiento.	2 horas
Sesión 18	Evaluación	Desarrollará el videojuego.	Integración final del proyecto.	2 horas
Sesión 19	Presentación	Expondrá el proyecto.	Presentación del videojuego.	2 horas
Sesión 20	Cierre	Entrega final.	Retroalimentación final.	2 horas

Material de apoyo:

Rigidbody2D en **Unity**
<https://docs.unity3d.com/Manual/class-Rigidbody2D.html>
 Uso de físicas en objetos 2D.

Colisiones en **Unity** **2D**
<https://docs.unity3d.com/Manual/CollidersOverview.html>
 Sistema de detección de colisiones.

UI en **Unity** **(Canvas)**
<https://docs.unity3d.com/Manual/class-Canvas.html>
 Creación de interfaces de usuario.

SceneManager
<https://docs.unity3d.com/ScriptReference/SceneManagement.SceneManager.html>
 Gestión de escenas en el juego.

Comunicación entre **scripts**
<https://docs.unity3d.com/Manual/Components.html>
 Interacción entre componentes.

Evaluación

La evaluación del módulo se realizará mediante una actividad práctica integradora y un cuestionario teórico, con el propósito de valorar la integración de todos los sistemas necesarios para el desarrollo de un videojuego completo.

La actividad práctica consistirá en el desarrollo de un nivel jugable con mecánicas, interfaz y flujo completo del juego.

El cuestionario teórico permitirá evaluar conceptos como físicas, colisiones, UI, gestión de escenas y buenas prácticas de programación.

La evaluación final se centrará en la actividad integradora.

Actividades de integración

Objetivo de la evaluación:

Valorar la capacidad de las y los usuarios para desarrollar un videojuego 2D completo en Unity integrando físicas, interfaz, lógica y flujo del juego de manera estructurada.

Actividad Integradora (Evaluación)

Objetivo:	Evaluar el desarrollo de un videojuego 2D completo y funcional.
Instrucciones:	Desarrolla la siguiente actividad de forma clara, organizada y funcional. La evidencia deberá presentarse en formato imagen (.jpg o .png) o PDF con un peso no mayor a 100 kb. Desarrolla un videojuego que incluya: • Movimiento lateral y salto con físicas reales • Enemigos funcionales • Sistema de vida y puntaje • Interfaz (HUD) dinámica • Menú principal funcional • Pantalla de victoria y derrota • Cambio de escenas funcional • Organización modular del código (valor 50.0) El sistema debe ser funcional, estructurado y jugable. Adjunta capturas del entorno, código y ejecución del videojuego.

Referencias

Microsoft. (2024). *C# Documentation*. Recuperado de:

<https://learn.microsoft.com/dotnet/csharp/>

Documentación oficial del lenguaje C#, que abarca desde fundamentos hasta programación orientada a objetos.

Microsoft. (2024). *Guía de programación en C#*. Recuperado de:
<https://learn.microsoft.com/dotnet/csharp/programming-guide/>
Explica estructuras, métodos, manejo de errores y buenas prácticas.

Unity Technologies. (2024). *Unity Manual*. Recuperado de:
<https://docs.unity3d.com/Manual/index.html>
Documentación oficial del motor Unity para desarrollo de videojuegos.

Unity Technologies. (2024). *Unity Scripting API*. Recuperado de:
<https://docs.unity3d.com/ScriptReference/>
Referencia completa para programación en C# dentro de Unity.

Unity Technologies. (2024). *Unity Learn*. Recuperado de:
<https://learn.unity.com/>
Plataforma de aprendizaje con cursos prácticos de desarrollo de videojuegos.

Albahari, J., & Albahari, B. (2023). *C# 12 in a Nutshell*. O'Reilly Media.
Libro de referencia completo sobre el lenguaje C# y sus aplicaciones.

Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6*. Apress.
Libro enfocado en programación profesional en C# con enfoque estructurado.

Murray, D. (2021). *Beginning C# Programming with Unity*. Packt Publishing.
Libro que integra C# con Unity aplicado al desarrollo de videojuegos.

Rogers, S. (2017). *Level Up! The Guide to Great Video Game Design* (2nd ed.). Wiley.
Guía sobre diseño de videojuegos, mecánicas y experiencia del usuario.

C.P. 01020, Alcaldía Álvaro Obregón. Ciudad de México

Subsistema de Educación Comunitaria "PILARES"
Dirección de Educación Participativa y Comunitaria

Coordinación General del Subsistema de Educación Comunitaria "PILARES"
Javier Ariel Hidalgo Ponce.

Dirección de Educación Participativa y Comunitaria
Luz del Alba Riande Méndez

Autora:
Diana Frine Zamano Macedonio



Reconocimiento-NoComercial-Compartirigual 4.0 Internacional

La propuesta de talleres "**Alfabetización Digital y Lógica Básica**" se distribuye como material de acceso abierto bajo los términos de la Licencia Creative Commons Atribución-NoComercial-Compartirigual 4.0 Internacional.

Se autoriza la copia, distribución, adaptación y comunicación pública de este contenido, total o parcial, siempre que se otorgue el crédito correspondiente a los autores y a la obra original "**Alfabetización Digital y Lógica Básica**", y se proporcione un enlace a la licencia.

Este material no puede ser utilizado con fines comerciales. Cualquier obra derivada o modificación de estos talleres debe ser distribuida bajo la misma licencia que rige el trabajo original.